

ACTIVITY_10 // PATH TRAVERSAL

Superfluous URL-Decode Bypass

File path traversal, traversal sequences stripped with superfluous URL-decode

CATEGORÍA	Path Traversal
PLATAFORMA	PortSwigger Web Security Academy
HERRAMIENTA	Burp Suite
MÓDULOS	Proxy + Repeater
VECTOR	Parámetro filename
ESTADO	SOLVED

Laboratorio realizado dentro del entorno académico autorizado de Web Security Academy. El objetivo consiste en recuperar el contenido de **/etc/passwd** desde el endpoint de imágenes, analizando un caso donde la validación ocurre antes de una decodificación URL adicional.

01 // LAB OBJECTIVE

La aplicación bloquea entradas que contienen secuencias de path traversal, pero posteriormente vuelve a decodificar el valor antes de utilizarlo. Esto permite representar temporalmente los caracteres peligrosos de una forma que no coincida con el filtro y hacer que reaparezcan después de la transformación.

VULNERABILITY SIGNAL

```
..%252f..%252f..%252fetc/passwd
```

02 // RESOLUTION PATH

01

Identificamos con Burp Suite una petición que obtiene una imagen de producto.

02

Enviamos la solicitud a Repeater y modificamos el parámetro filename con la representación codificada requerida. En este documento se conserva como: `..%252f..%252f..%252fetc/passwd`.

03

Reenviamos la petición y verificamos que la respuesta contiene el archivo `/etc/passwd`, confirmando el bypass.

03 // EVIDENCE RUN - STEP 01

Desde Proxy > HTTP history localizamos una solicitud GET al endpoint /image. La petición seleccionada usa filename=61.jpg, por lo que confirmamos que el recurso solicitado se controla mediante un parámetro modificable.

The screenshot shows the Burp Suite interface. The top menu bar includes Burp, Project, Intruder, Repeater, Window, and Help. Below it, a sub-menu bar has Dashboard, Target, Proxy (selected), Intruder, Repeater, Sequencer, Decoder, Comparer, Logger, and Extender. Under the Proxy menu, there are Project options, User options, and Learn. The main toolbar shows Intercept, HTTP history (selected), WebSockets history, and Options. A filter bar at the top of the history table reads 'Filter: Hiding CSS, image and general binary content'. The history table lists 13 requests, with request 1374 highlighted in orange. The table columns are #, Host, Method, URL, Params, Edited, and Status. Below the history table, the 'Request' tab is active, showing the raw HTTP request details. The request is a GET to /image?filename=61.jpg. The host is 0a26007e03a6b5aec092dba8004200fe.web-security-academy.net. The request includes cookies, user-agent, and other headers. The bottom of the interface shows a search bar with '0 matches'.

#	Host	Method	URL	Params	Edited	Status
1379	https://0a26007e03a6b5aec092...	GET	/academyLabHeader			
1378	https://0a26007e03a6b5aec092...	GET	/resources/labheader/images/ps-lab-not...			
1377	https://0a26007e03a6b5aec092...	GET	/resources/labheader/images/logoAcade...			
1376	https://0a26007e03a6b5aec092...	GET	/image?filename=19.jpg	✓		
1375	https://0a26007e03a6b5aec092...	GET	/image?filename=25.jpg	✓		
1374	https://0a26007e03a6b5aec092...	GET	/image?filename=61.jpg	✓		
1373	https://0a26007e03a6b5aec092...	GET	/image?filename=18.jpg	✓		
1372	https://0a26007e03a6b5aec092...	GET	/image?filename=5.jpg	✓		
1371	https://0a26007e03a6b5aec092...	GET	/image?filename=40.jpg	✓		
1370	https://0a26007e03a6b5aec092...	GET	/image?filename=46.jpg	✓		
1369	https://0a26007e03a6b5aec092...	GET	/image?filename=43.jpg	✓		
1368	https://0a26007e03a6b5aec092...	GET	/image?filename=45.jpg	✓		
1366	https://0a26007e03a6b5aec092...	GET	/image?filename=50.jpg	✓		
1365	https://0a26007e03a6b5aec092...	GET	/image?filename=15.jpg	✓		

Request

Pretty Raw Hex

```
1 GET /image?filename=61.jpg HTTP/1.1
2 Host: 0a26007e03a6b5aec092dba8004200fe.web-security-academy.net
3 Cookie: session=atAnJWdLNE8i4grLutW8jLR10VYcAVoD
4 Sec-Ch-Ua: "Chromium";v="105", "Not)A;Brand";v="8"
5 Sec-Ch-Ua-Mobile: ?0
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/105.0.5195.102 Safari/537.36
7 Sec-Ch-Ua-Platform: "Linux"
8 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: no-cors
```

0 matches

Figura 1. Historial HTTP con la solicitud de imagen seleccionada.

STEP 02A // FIRST ENCODED ATTEMPT

En Repeater sustituimos el valor original por una representación codificada del recorrido hacia el archivo objetivo. La respuesta 400 Bad Request con el mensaje No such file muestra que esa representación todavía no produce la ruta que la aplicación utilizará finalmente.

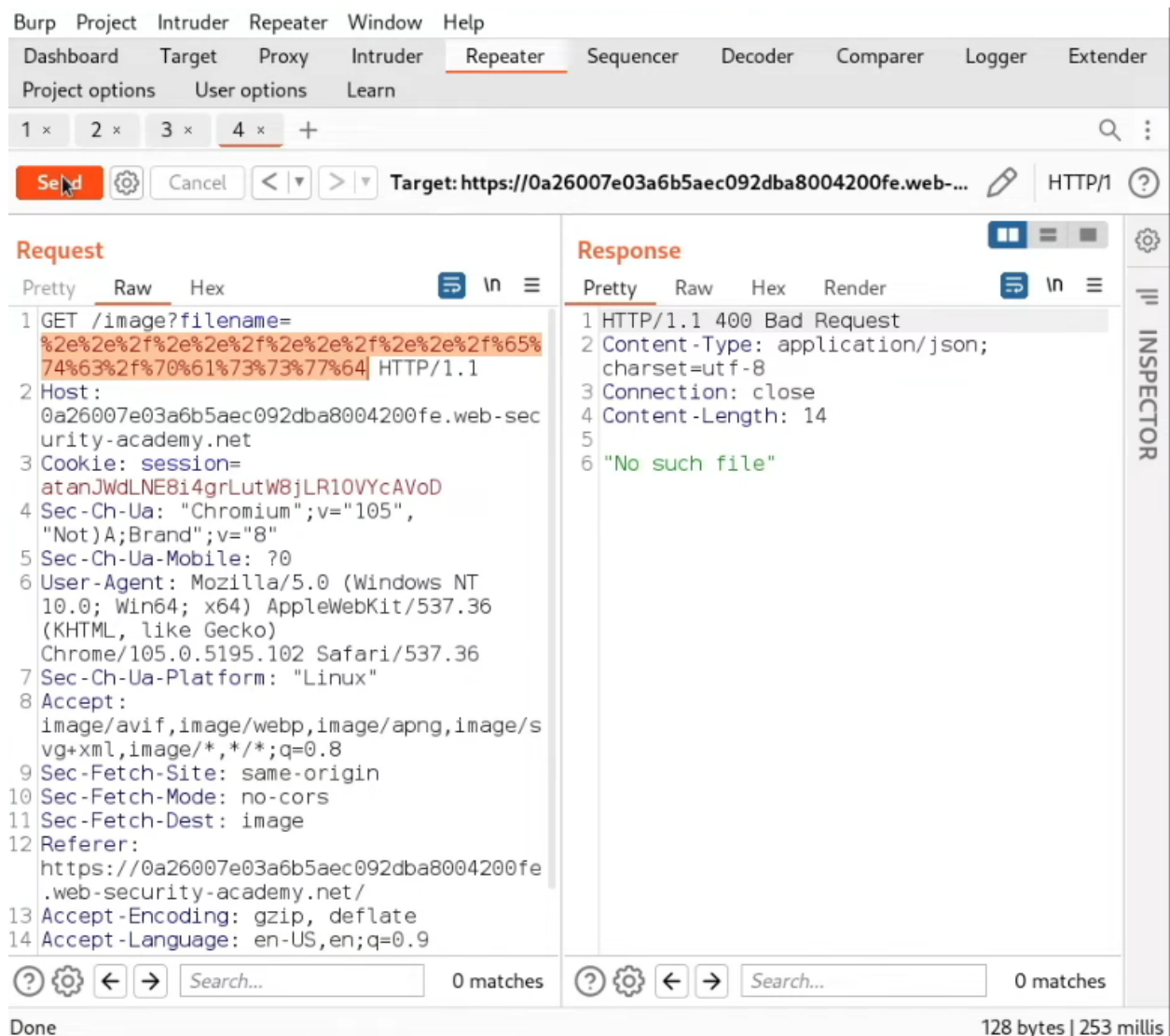


Figura 2. Primer intento codificado y respuesta de error.

STEP 02B // EXTRA ENCODING LAYER

Seleccionamos el valor del parámetro y utilizamos Convert selection > URL > URL-encode all characters. Con esto añadimos una capa extra de codificación antes de reenviar la petición.

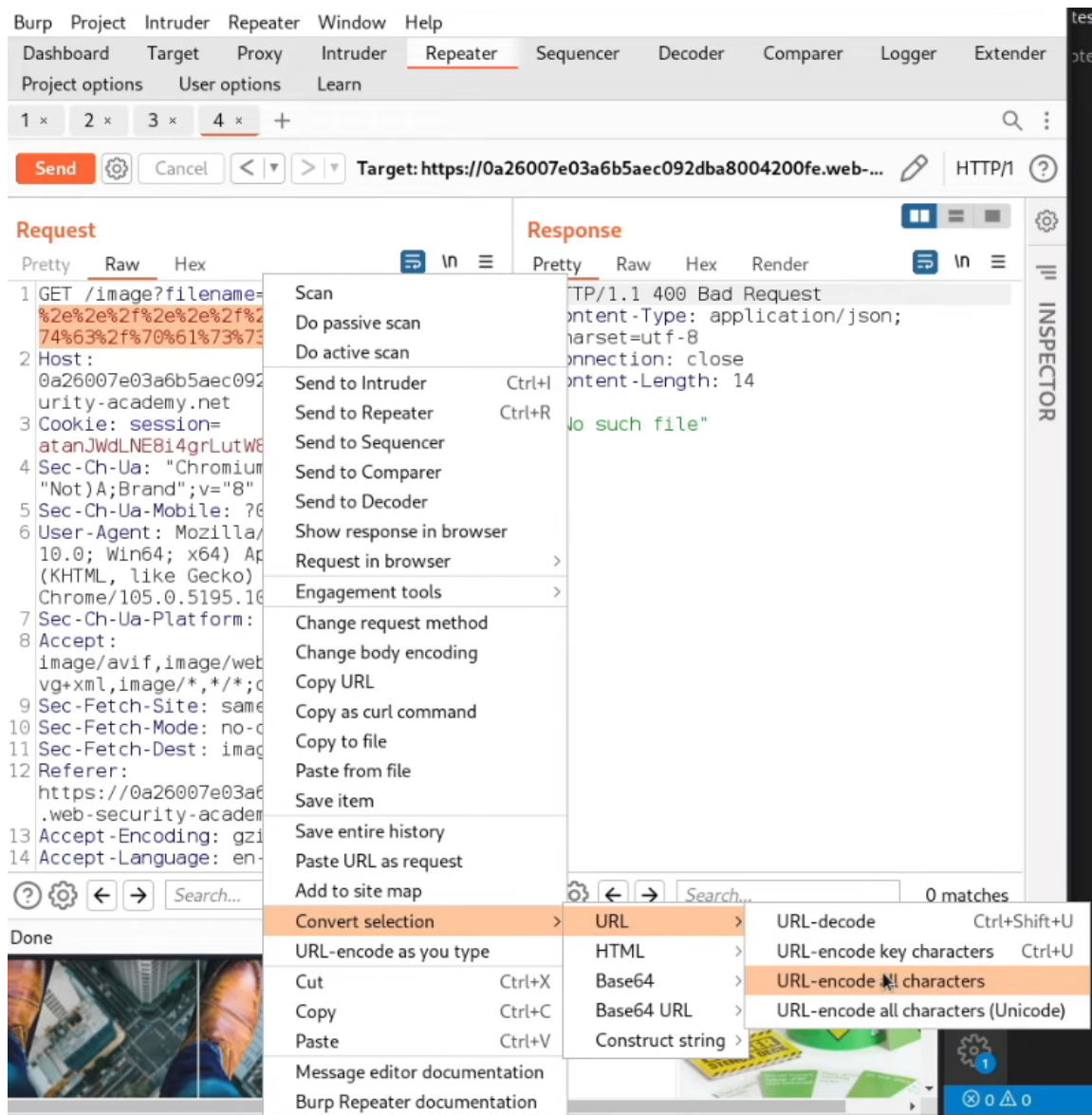


Figura 3. Aplicación de URL-encode all characters desde Burp Repeater.

ENCODED VALUE ..

%252f..%252f..%252fetc/passwd

STEP 03 // BYPASS CONFIRMED

Con la nueva representación, el servidor responde HTTP/1.1 200 OK y devuelve entradas del archivo `/etc/passwd`. Esto confirma que una etapa posterior de decodificación reconstruye la secuencia de traversal después del filtro.

[illegible]

Figura 4. Respuesta 200 OK con contenido de /etc/passwd.

FINAL // VALIDATION

Web Security Academy reconoce el resultado requerido y marca el laboratorio como Solved, dejando constancia de que la práctica fue completada correctamente dentro del entorno autorizado.

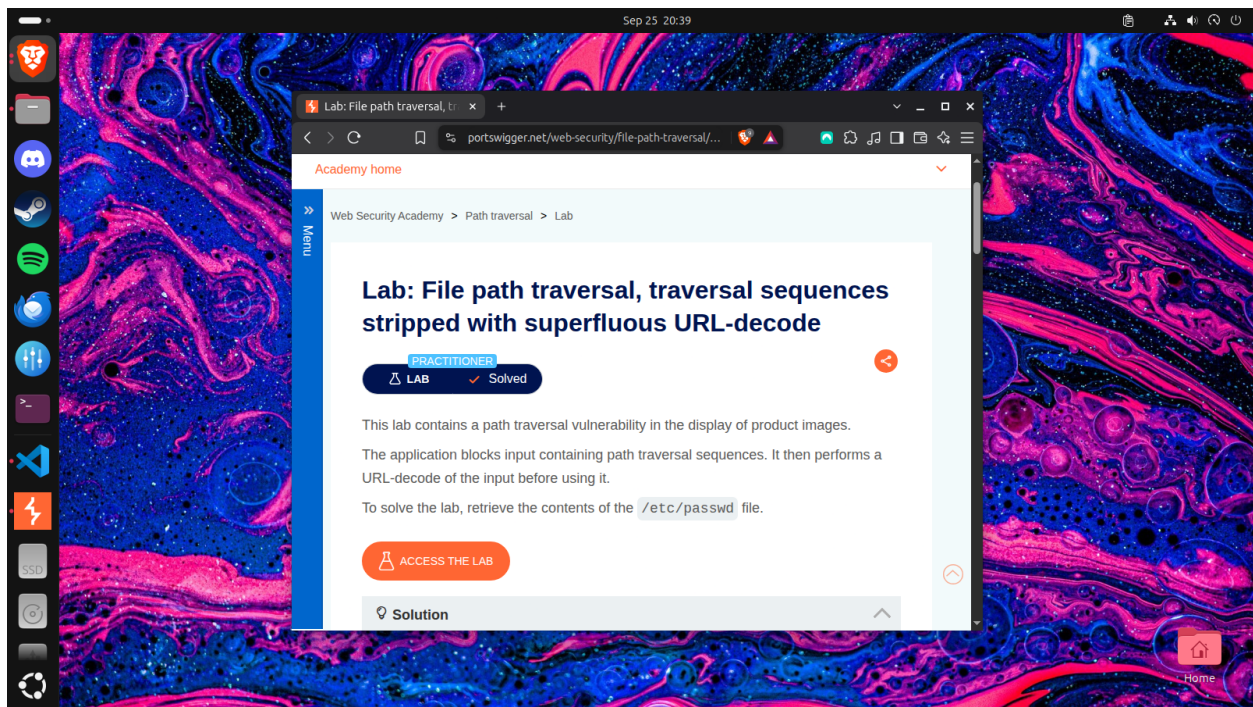


Figura 5. Laboratorio marcado como resuelto.

04 // TECHNICAL ANALYSIS

01 // VALIDACIÓN ANTES DE NORMALIZAR

El filtro examina una representación intermedia de la entrada antes de que terminen todas las transformaciones.

02 // DECODIFICACIÓN ADICIONAL

La capa extra de URL-encoding evita que el traversal aparezca directamente durante la primera validación; la decodificación posterior lo reconstruye.

03 // IMPACTO

Un atacante puede salir del directorio previsto para imágenes y leer archivos locales accesibles para el proceso web.

04 // MITIGACIÓN

La entrada debe decodificarse y canonicalizarse antes de validarla. Además, conviene usar una allowlist de archivos o identificadores y comprobar que la ruta final permanezca dentro del directorio autorizado.

RESULT

El laboratorio demuestra que filtrar una representación de la entrada y transformarla después puede reintroducir patrones que la validación pretendía bloquear. La seguridad debe aplicarse sobre la forma final y canonicalizada de la ruta.